

Driver ADS — Plano Completo de Implementação do Sistema de QR Code Rastreável

1. Objetivo da Feature

Implementar na plataforma Driver ADS um sistema proprietário de QR Codes rastreáveis para campanhas físicas impressas em anúncios de encosto veicular.

O QR Code impresso deve:

1. Não expirar.
2. Apontar para uma URL permanente do domínio `driverads.com.br`.
3. Registrar cada leitura/acesso.
4. Redirecionar o usuário para o destino configurado da campanha.
5. Alimentar métricas no dashboard do anunciante.
6. Permitir aplicação automática do QR real nas artes finais antes da impressão.
7. Ser adaptado à estrutura atual da plataforma, evitando criação de tabelas, serviços ou lógicas duplicadas.

A lógica estratégica é:

QR Code impresso fixo → URL rastreável da Driver ADS → registro do scan → redirecionamento para WhatsApp/site/landing page → métricas no dashboard

A regra mais importante:

O QR Code não deve apontar diretamente para o WhatsApp ou site do anunciante.
O QR Code deve apontar para uma URL própria da Driver ADS.

Exemplo:

`https://driverads.com.br/q/oticas-diniz-001`

Essa URL registra o evento e depois redireciona para o destino final.

2. Contexto Operacional

Hoje as artes podem ser criadas com IA ou manualmente. Algumas imagens geradas já vêm com QR Code fictício dentro do layout.

Esse QR Code fictício não deve ser usado para impressão final.

O fluxo correto deve ser:

Arte base gerada → sistema gera QR real → sistema aplica QR real na arte → arte final é exportada → gráfica imprime

A arte base pode vir com um espaço reservado para QR Code. Se já existir um QR fictício na arte, o sistema deve cobrir essa área com um fundo branco e inserir o QR real por cima.

3. Premissa Arquitetural

Não utilizar n8n nesta fase.

O sistema deve ser implementado diretamente na plataforma atual da Driver ADS.

Antes de criar qualquer nova tabela ou módulo, o Codex deve auditar a estrutura atual do projeto e do banco de dados para identificar entidades existentes que já possam representar:

- anunciantes;
- campanhas;
- motoristas;
- veículos;
- peças/anúncios;
- instalações;
- assets/imagens;
- métricas;
- eventos;
- leads;
- usuários;
- permissões.

A implementação deve reaproveitar o máximo possível da estrutura atual.

Não criar tabelas novas se já existir uma tabela equivalente.

Não criar um segundo modelo de campanha se já existir uma entidade de campanha funcional.

Não criar uma tabela de anunciantes se já existir uma tabela de empresas, clientes, advertisers, accounts ou organizations que cumpra esse papel.

Não criar uma tabela de veículos se já existir uma tabela de vehicles, cars, driver_vehicles ou similar.

Não criar uma tabela de métricas se já existir estrutura de event tracking ou analytics reaproveitável.

A criação de novas tabelas só deve ocorrer quando a estrutura atual não cobrir os requisitos mínimos da feature.

4. Orientação Inicial para o Codex

Antes de implementar, executar uma auditoria do projeto.

4.1. Auditar estrutura do código

Identificar:

- framework utilizado;
- rotas de API existentes;
- camada de autenticação;
- client Supabase ou outro client de banco;
- estrutura de services;
- estrutura de repositories;
- padrões de migrations;
- padrões de naming;
- storage/buckets existentes;
- componentes de dashboard;
- módulos de campanhas;
- módulos de usuários/anunciantes;
- módulos de motoristas/veículos;
- módulos de upload/exportação de arte.

4.2. Auditar banco de dados

Verificar tabelas existentes com nomes semelhantes a:

```
advertisers
clients
customers
companies
organizations
accounts
campaigns
ads
ad_assets
creatives
drivers
vehicles
driver_vehicles
placements
installations
campaign_vehicles
campaign_assets
events
analytics_events
metrics
leads
```

```
qr_codes  
scan_events
```

Se possível, gerar um resumo antes de migrar:

```
select table_name  
from information_schema.tables  
where table_schema = 'public'  
order by table_name;
```

Depois verificar colunas das tabelas candidatas:

```
select table_name, column_name, data_type  
from information_schema.columns  
where table_schema = 'public'  
order by table_name, ordinal_position;
```

O Codex deve produzir um pequeno relatório interno antes de aplicar migrations:

1. Tabelas existentes reaproveitadas.
2. Tabelas novas necessárias.
3. Colunas novas necessárias.
4. Funções novas necessárias.
5. Rotas novas necessárias.
6. Buckets novos necessários.

5. Modelo Conceitual da Feature

Existem 5 blocos principais:

1. QR Engine
2. Redirect & Tracking Engine
3. Print Asset Engine
4. Metrics Engine
5. Dashboard Integration

5.1. QR Engine

Responsável por:

- criar slug rastreável;
- gerar URL permanente;
- gerar imagem do QR Code;

- vincular QR a campanha, peça e/ou veículo;
- armazenar status do QR.

5.2. Redirect & Tracking Engine

Responsável por:

- receber acesso na URL /q/:slug;
- encontrar o QR correspondente;
- validar status;
- registrar evento de leitura;
- redirecionar para o destino final;
- nunca retornar 404 para QR impresso sem fallback.

5.3. Print Asset Engine

Responsável por:

- receber arte base;
- gerar QR real;
- aplicar o QR na posição correta da arte;
- exportar arte final para impressão;
- salvar preview e final no Storage;
- permitir download pelo painel interno.

5.4. Metrics Engine

Responsável por:

- consolidar leituras totais;
- leituras únicas estimadas;
- leituras por dia;
- leituras por campanha;
- leituras por veículo/posição quando houver;
- leituras por período;
- cliques/redirecionamentos;
- eventos de lead se existirem.

5.5. Dashboard Integration

Responsável por:

- exibir métricas para o anunciante;
- exibir métricas internas para operação;
- filtrar por campanha;
- filtrar por período;

- filtrar por peça/veículo, se aplicável;
- permitir baixar arte final.

6. Decisão Importante: QR por Campanha ou QR por Veículo?

Existem dois modelos possíveis.

Modelo A — 1 QR por campanha

Mais simples.

Campanha Óticas Diniz → 1 QR Code único → todos os carros usam o mesmo QR

Vantagens:

- implementação mais rápida;
- impressão mais simples;
- menos arquivos finais;
- ideal para MVP.

Desvantagens:

- não mede performance por carro;
- não mede performance por motorista;
- não diferencia local de veiculação;
- menos inteligência operacional.

Modelo B — 1 QR por placement/veículo

Mais poderoso.

Campanha Óticas Diniz → QR A no carro 1
Campanha Óticas Diniz → QR B no carro 2
Campanha Óticas Diniz → QR C no carro 3

Vantagens:

- mede performance por veículo;
- mede performance por motorista;
- permite ranking de carros;
- permite otimizar distribuição;
- aumenta muito o valor percebido para o anunciante.

Desvantagens:

- exige gerar várias artes finais;
- exige controle operacional maior;
- exige vincular QR à instalação.

Recomendação

Construir a arquitetura já preparada para o Modelo B, mas permitir operar no Modelo A no MVP.

Ou seja:

```
qr_codes.campaign_id = obrigatório  
qr_codes.placement_id = opcional no MVP
```

No começo, se ainda não houver vínculo com veículos, o QR pode ser da campanha.

Depois, quando a operação estiver mais madura, cada instalação recebe um QR único.

7. Estrutura de Dados Recomendada

Abaixo estão sugestões conceituais. O Codex deve adaptar ao banco atual.

7.1. Se já existir tabela de campanhas

Reaproveitar a tabela existente.

Adicionar apenas colunas se necessário:

```
alter table campaigns  
add column if not exists default_destination_url text,  
add column if not exists default_destination_type text,  
add column if not exists tracking_enabled boolean default true;
```

Usar nomes compatíveis com o padrão atual do banco.

7.2. Tabela de QR Codes

Criar apenas se não existir equivalente.

Nome sugerido:

```
qr_codes
```

SQL sugerido:

```
create table if not exists qr_codes (  
  id uuid primary key default gen_random_uuid(),  
  
  campaign_id uuid not null,  
  placement_id uuid null,  
  
  slug text not null unique,  
  tracking_url text not null unique,  
  
  destination_url text not null,  
  destination_type text null,  
  
  status text not null default 'active',  
  
  qr_image_path text null,  
  qr_svg_path text null,  
  
  metadata jsonb not null default '{}'::jsonb,  
  
  created_by uuid null,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Observações:

- campaign_id deve referenciar a tabela real de campanhas.
- placement_id deve referenciar a tabela real de instalações/veiculações, se existir.
- Se o banco atual usa outro nome para campanha ou placement, adaptar.
- Não criar foreign key antes de confirmar nomes reais e padrão atual do banco.

Índices sugeridos:

```
create index if not exists idx_qr_codes_campaign_id on qr_codes(campaign_id);  
create index if not exists idx_qr_codes_placement_id on  
qr_codes(placement_id);  
create index if not exists idx_qr_codes_status on qr_codes(status);  
create unique index if not exists idx_qr_codes_slug_unique on qr_codes(slug);
```

7.3. Tabela de eventos de scan

Criar apenas se não existir uma tabela equivalente de eventos/analytics.

Nome sugerido:

qr_scan_events

SQL sugerido:

```
create table if not exists qr_scan_events (  
  id uuid primary key default gen_random_uuid(),  
  
  qr_code_id uuid not null,  
  campaign_id uuid not null,  
  placement_id uuid null,  
  
  scanned_at timestampz not null default now(),  
  
  ip_hash text null,  
  user_agent text null,  
  device_type text null,  
  browser text null,  
  os text null,  
  
  referrer text null,  
  country text null,  
  region text null,  
  city text null,  
  
  redirected_to text null,  
  is_bot boolean not null default false,  
  
  metadata jsonb not null default '{}'::jsonb,  
  
  created_at timestampz not null default now()  
);
```

Índices sugeridos:

```
create index if not exists idx_qr_scan_events_qr_code_id on  
qr_scan_events(qr_code_id);  
create index if not exists idx_qr_scan_events_campaign_id on  
qr_scan_events(campaign_id);  
create index if not exists idx_qr_scan_events_placement_id on  
qr_scan_events(placement_id);  
create index if not exists idx_qr_scan_events_scanned_at on  
qr_scan_events(scanned_at);  
create index if not exists idx_qr_scan_events_campaign_date on  
qr_scan_events(campaign_id, scanned_at);
```

7.4. Tabela de assets finais de impressão

Criar apenas se não existir estrutura de assets/creatives.

Nome sugerido:

campaign_print_assets

SQL sugerido:

```
create table if not exists campaign_print_assets (  
  id uuid primary key default gen_random_uuid(),  
  
  campaign_id uuid not null,  
  qr_code_id uuid null,  
  placement_id uuid null,  
  
  base_art_path text not null,  
  final_art_path text null,  
  
  layout_template text null,  
  
  qr_x integer null,  
  qr_y integer null,  
  qr_width integer null,  
  qr_height integer null,  
  
  output_width integer null,  
  output_height integer null,  
  
  format text not null default 'png',  
  status text not null default 'draft',  
  
  metadata jsonb not null default '{} '::jsonb,  
  
  created_by uuid null,  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

8. URL Permanente do QR

Usar sempre o domínio real:

https://driverads.com.br/q/{slug}

Exemplos:

```
https://driverads.com.br/q/diniz-001  
https://driverads.com.br/q/voah-001  
https://driverads.com.br/q/subway-centro-001
```

8.1. Regras do slug

O slug deve ser:

- curto;
- legível;
- único;
- sem acentos;
- sem espaços;
- estável;
- não reciclável.

Exemplo recomendado:

```
{brand}-{campaign_short_code}-{random}
```

Exemplo:

```
diniz-lentes-hoya-a7k2
```

Não reaproveitar slug antigo em campanha nova.

9. Endpoint de Redirecionamento

Criar rota pública:

```
GET /q/:slug
```

Essa rota deve:

1. receber o slug;
2. buscar qr_codes.slug;
3. validar se existe;
4. validar status;
5. registrar scan;

6. redirecionar para destination_url;
7. se estiver inativo/encerrado, redirecionar para fallback seguro.

9.1. Pseudocódigo

```
export async function handleQrRedirect(request, params) {
  const slug = params.slug;

  const qr = await findQrBySlug(slug);

  if (!qr) {
    return redirect('https://driverads.com.br/campanha-indisponivel');
  }

  const destination = resolveDestination(qr);

  await recordQrScan({
    qrCodeId: qr.id,
    campaignId: qr.campaign_id,
    placementId: qr.placement_id,
    request,
    redirectedTo: destination,
  });

  return redirect(destination, 302);
}
```

9.2. Nunca fazer

- nunca retornar erro técnico para usuário final;
- nunca deixar QR impresso cair em 404 seco;
- nunca expor dados internos no redirect;
- nunca depender do front para registrar scan;
- nunca usar QR direto para wa.me sem tracking.

9.3. Fallback recomendado

Se campanha estiver inativa, usar:

<https://driverads.com.br/campanha-encerrada>

Ou, melhor:

<https://driverads.com.br/ofertas>

Ou ainda:

uma landing customizada da campanha encerrada.

10. Destinos de Campanha

A campanha pode redirecionar para diferentes tipos de destino.

Tipos sugeridos:

```
whatsapp  
website  
landing_page  
form  
menu  
coupon  
custom_url
```

10.1. WhatsApp

Exemplo de destino:

```
https://wa.me/5548999999999?  
text=Olá%2C%20vi%20sua%20campanha%20pela%20Driver%20ADS%20e%20quero%20saber%20mais.
```

O texto deve conter referência rastreável humana:

```
Olá, vi sua campanha pela Driver ADS e quero saber mais. Código: DINIZ001
```

Mesmo que o WhatsApp não devolva evento de conversão, o atendente e o anunciante conseguem reconhecer a origem.

10.2. Landing Page

Exemplo:

```
https://cliente.com/oferta?  
utm_source=driverads&utm_medium=backseat&utm_campaign=diniz-lentes-hoya
```

10.3. Página intermediária Driver ADS

Para mais controle, futuramente a Driver ADS pode abrir uma página intermediária própria antes do WhatsApp:

```
driverads.com.br/l/diniz-lentes-hoya
```

Essa página pode registrar:

- carregamento;
- clique no botão;
- tempo na página;
- conversão;
- origem do QR.

Mas no MVP, o redirect direto pós-tracking é suficiente.

11. Geração do QR Code

11.1. Biblioteca recomendada

Se o projeto usa Node/Next/API server:

```
npm install qrcode
```

Outras libs úteis:

```
npm install nanoid ua-parser-js sharp
```

Dependências sugeridas:

```
qrcode      → gerar PNG/SVG do QR
nanoid      → gerar slugs curtos
ua-parser-js → classificar device/browser/OS
sharp       → compor QR na arte final
```

Se o projeto já usa outras libs equivalentes, reaproveitar as existentes.

11.2. Configuração recomendada do QR

Para impressão em carro, usar QR simples e confiável.

```
const qrOptions = {
  errorCorrectionLevel: 'H',
  type: 'png',
  margin: 2,
  color: {
    dark: '#000000',
```

```
    light: '#FFFFFF',  
  },  
  width: 900,  
};
```

Regras:

- QR preto sobre fundo branco;
- não usar QR estilizado no MVP;
- manter quiet zone/margem;
- usar alta resolução;
- testar leitura em celular real antes de imprimir.

12. Tamanho físico recomendado do QR

Como o cartaz físico da Driver ADS tem proporção aproximada de 15x20 cm, o QR precisa ter tamanho confortável.

Recomendação:

Tamanho físico mínimo do QR: 3 cm
Tamanho ideal: 3,5 cm a 4,5 cm

Para impressão 300 DPI:

3 cm $\approx$ 354 px
4 cm $\approx$ 472 px
4,5 cm $\approx$ 531 px

Recomendação prática:

Gerar QR com pelo menos 900x900 px.
Aplicar na arte final com tamanho visual entre 450 e 550 px se a arte estiver em 300 DPI.

Se o layout for 20x15 cm a 300 DPI:

20 cm $\approx$ 2362 px
15 cm $\approx$ 1772 px

O Codex deve verificar o tamanho real da arte exportada antes de posicionar o QR.

13. Print Asset Engine

13.1. Objetivo

Gerar arte final para impressão com QR real aplicado.

Entrada:

- campanha;
- arte base;
- QR Code;
- template de posição;
- formato de exportação.

Saída:

- arte final PNG/JPG;
- opcionalmente PDF;
- preview no painel;
- arquivo salvo no Storage.

13.2. Fluxo

1. Usuário cria campanha.
2. Usuário configura destino do QR.
3. Sistema gera QR rastreável.
4. Usuário sobe a arte base.
5. Usuário escolhe template de layout.
6. Sistema aplica QR real na arte.
7. Sistema gera preview.
8. Usuário testa o QR.
9. Usuário baixa arte final para impressão.

13.3. Template de posição

Criar configuração de layout em JSON, em vez de fixar coordenadas no código.

Exemplo:

```
{
  "name": "driverads_landscape_right_qr_v1",
  "description": "Arte horizontal com QR no bloco branco lateral direito",
  "canvas": {
    "width": 1600,
    "height": 900
  },
}
```



```

    "qr": {
      "x": 1230,
      "y": 355,
      "width": 260,
      "height": 260,
      "background": "#FFFFFF",
      "padding": 18,
      "borderRadius": 12
    }
  }
}

```

Atenção:

Essas coordenadas são exemplo.
O Codex deve ajustar de acordo com o tamanho real das artes da Driver ADS.

13.4. Aplicação com sharp

Pseudocódigo:

```

import sharp from 'sharp';
import QRCode from 'qrcode';

async function generateFinalPrintAsset({
  baseImageBuffer,
  trackingUrl,
  qrBox,
  outputFormat = 'png',
}) {
  const qrPngBuffer = await QRCode.toBuffer(trackingUrl, {
    errorCorrectionLevel: 'H',
    margin: 2,
    color: {
      dark: '#000000',
      light: '#FFFFFF',
    },
    width: qrBox.width - qrBox.padding * 2,
  });

  const whiteBoxBuffer = await sharp({
    create: {
      width: qrBox.width,
      height: qrBox.height,
      channels: 4,
      background: '#FFFFFF',
    },
  })
  .png()
  .toBuffer();

```

```

const qrWithBackground = await sharp(whiteBoxBuffer)
  .composite([
    {
      input: qrPngBuffer,
      left: qrBox.padding,
      top: qrBox.padding,
    },
  ])
  .png()
  .toBuffer();

const finalBuffer = await sharp(baseImageBuffer)
  .composite([
    {
      input: qrWithBackground,
      left: qrBox.x,
      top: qrBox.y,
    },
  ])
  .toFormat(outputFormat)
  .toBuffer();

return finalBuffer;
}

```

13.5. Importante sobre runtime

`sharp` deve rodar em ambiente Node/server, não em edge runtime se houver incompatibilidade com dependências nativas.

Se o projeto for Next.js:

```
export const runtime = 'nodejs';
```

Não usar `runtime = 'edge'` para composição com `sharp`.

14. Storage de Arquivos

Se já existir bucket de imagens/assets, reaproveitar.

Caso precise criar, sugerir buckets:

```

campaign-base-assets
campaign-final-assets
qr-code-assets

```

Ou um único bucket:

```
driverads-assets
```

Com paths organizados:

```
campaigns/{campaignId}/base/{assetId}.png  
campaigns/{campaignId}/final/{assetId}.png  
campaigns/{campaignId}/qr/{qrCodeId}.png
```

Exemplo:

```
campaigns/diniz-001/base/original.png  
campaigns/diniz-001/final/diniz-car-001.png  
campaigns/diniz-001/qr/diniz-car-001.png
```

15. Painel Interno de Geração

Criar ou adaptar uma tela interna para operação.

Campos necessários:

```
- anunciante;  
- campanha;  
- destino do QR;  
- tipo de destino;  
- upload da arte base;  
- seleção do template;  
- prévia do QR;  
- prévia da arte final;  
- botão gerar;  
- botão testar QR;  
- botão baixar final;  
- botão ativar campanha.
```

15.1. Botão “Testar QR”

Antes de liberar para impressão, o sistema deve obrigar validação.

Status possíveis:

```
draft  
qr_generated
```

```
asset_generated
tested
approved_for_print
printed
installed
active
paused
completed
archived
```

MVP pode usar menos status, mas precisa ter pelo menos:

```
draft
active
paused
completed
```

Para asset:

```
draft
generated
approved
```

16. Métricas

16.1. Métricas mínimas para MVP

No dashboard do anunciante:

- total de leituras do QR;
- leituras únicas estimadas;
- leituras por dia;
- leituras por campanha;
- últimas leituras;
- campanha ativa/inativa;
- destino configurado;
- período da campanha.

16.2. Métricas para operação interna

No dashboard interno:

- scans por QR;
- scans por campanha;

- scans por veículo/placement;
- scans por motorista;
- ranking de QR Codes;
- veículos ativos;
- peças instaladas;
- campanhas com melhor performance;
- horários com mais leitura.

16.3. Métricas futuras

- custo por scan;
- custo por lead;
- scan por bairro/região estimada;
- comparativo de campanhas;
- funil: escaneou → abriu WhatsApp → lead qualificado → venda;
- alertas de performance;
- relatório automático por período.

17. Views SQL para Dashboard

Se não houver camada de analytics existente, criar views.

17.1. View de resumo por campanha

```
create or replace view campaign_qr_metrics as
select
  q.campaign_id,
  count(e.id) as total_scans,
  count(distinct e.ip_hash) as estimated_unique_scans,
  min(e.scanned_at) as first_scan_at,
  max(e.scanned_at) as last_scan_at
from qr_codes q
left join qr_scan_events e on e.qr_code_id = q.id
group by q.campaign_id;
```

17.2. View de scans por dia

```
create or replace view campaign_qr_daily_metrics as
select
  campaign_id,
  date_trunc('day', scanned_at)::date as scan_date,
  count(*) as total_scans,
  count(distinct ip_hash) as estimated_unique_scans
from qr_scan_events
group by campaign_id, date_trunc('day', scanned_at)::date;
```

17.3. View por QR Code

```
create or replace view qr_code_metrics as
select
  q.id as qr_code_id,
  q.slug,
  q.campaign_id,
  q.placement_id,
  q.status,
  count(e.id) as total_scans,
  count(distinct e.ip_hash) as estimated_unique_scans,
  max(e.scanned_at) as last_scan_at
from qr_codes q
left join qr_scan_events e on e.qr_code_id = q.id
group by q.id, q.slug, q.campaign_id, q.placement_id, q.status;
```

O Codex deve adaptar nomes e permissões conforme o padrão atual do projeto.

18. Privacidade e LGPD

Não armazenar IP puro se não for necessário.

Usar hash com salt server-side.

Exemplo conceitual:

```
function hashIp(ip: string, salt: string) {
  return sha256(`${salt}:${ip}`);
}
```

Armazenar:

```
ip_hash
user_agent
device_type
browser
os
city aproximada, se disponível
```

Evitar armazenar:

```
IP bruto permanente
dados pessoais sensíveis
informações não necessárias
```

Se for capturar lead posteriormente, separar evento anônimo de scan do evento identificado de lead.

19. Segurança

19.1. Rate limit

Adicionar proteção para evitar spam de scan.

Opções:

- limitar scans repetidos do mesmo ip_hash + qr_code_id em curto período;
- registrar todos os eventos, mas marcar duplicados;
- criar campo metadata.duplicate_window = true;

Exemplo:

Se o mesmo IP/device escanear o mesmo QR 10 vezes em 1 minuto, contar como evento bruto, mas não como unique scan.

19.2. Bot detection

Marcar bots via user-agent.

```
Googlebot  
Bingbot  
WhatsApp preview  
Facebook crawler  
Instagram crawler
```

Criar `is_bot = true`.

No dashboard do anunciante, por padrão, mostrar métricas sem bots.

19.3. Open redirect

Não permitir que qualquer pessoa crie destino malicioso sem controle.

Validar destination_url ao criar campanha.

No painel administrativo, permitir somente usuários autorizados.

19.4. Status do QR

Status sugeridos:

active
paused
completed
archived
blocked

Se `blocked`, redirecionar para página segura da Driver ADS.

20. Integração com Front-end

20.1. Dashboard do anunciante

Adicionar seção:

Performance do QR Code

Cards:

Leituras totais
Leituras únicas estimadas
Leituras hoje
Leituras últimos 7 dias
Campanha ativa
Destino

Gráficos:

Scans por dia
Scans por horário
Scans por QR/veículo, se houver

Tabela:

Data/hora
Campanha
QR
Dispositivo
Cidade estimada
Destino

20.2. Painel interno

Adicionar tela:

Campanhas → QR & Impressão

Ações:

Gerar QR
Gerar arte final
Testar QR
Baixar PNG
Baixar PDF
Ativar campanha
Pausar QR
Alterar destino

21. Endpoints/API Contracts

Adaptar ao framework atual.

21.1. Criar QR

`POST /api/qr-codes`

Payload:

```
{
  "campaignId": "uuid",
  "placementId": "uuid opcional",
  "destinationUrl": "https://...",
  "destinationType": "whatsapp"
}
```

Resposta:

```
{
  "id": "uuid",
  "slug": "diniz-lentes-a7k2",
  "trackingUrl": "https://driverads.com.br/q/diniz-lentes-a7k2",
  "qrImagePath": "campaigns/.../qr.png"
}
```

21.2. Redirect público

`GET /q/:slug`

Resposta:

```
302 redirect para destino final
```

21.3. Gerar arte final

```
POST /api/campaign-assets/generate-final
```

Payload:

```
{
  "campaignId": "uuid",
  "qrCodeId": "uuid",
  "baseArtPath": "campaigns/.../base.png",
  "layoutTemplate": "driverads_landscape_right_qr_v1"
}
```

Resposta:

```
{
  "finalArtPath": "campaigns/.../final.png",
  "previewUrl": "https://..."
}
```

21.4. Métricas

```
GET /api/campaigns/:id/qr-metrics?from=2026-06-01&to=2026-06-30
```

Resposta:

```
{
  "totalScans": 1284,
  "uniqueScans": 842,
  "daily": [
    {
      "date": "2026-06-01",
      "scans": 42,
      "uniqueScans": 31
    }
  ],
  "byQrCode": [
    {
      "qrCodeId": "uuid",
      "slug": "diniz-001",

```

```
    "scans": 200
  }
]
}
```

22. Ordem de Implementação Recomendada

Fase 0 — Auditoria

Objetivo:

Entender estrutura atual antes de criar qualquer coisa.

Tarefas:

- mapear tabelas existentes;
- mapear rotas existentes;
- mapear storage existente;
- mapear entidades de campanha/anunciante/veículo;
- decidir o mínimo de novas tabelas/colunas.

Entregável:

Relatório de adaptação: o que será reaproveitado e o que será criado.

Fase 1 — QR Tracking MVP

Objetivo:

Criar QR permanente e rastreável.

Tarefas:

- criar/adaptar tabela de qr_codes;
- criar/adaptar tabela de scan events;
- criar endpoint /q/:slug;
- registrar scan;
- redirecionar para destino;
- criar função de hash de IP;
- adicionar bot detection básico;
- testar QR real.

Critério de aceite:

Ao acessar <https://driverads.com.br/q/teste>, o sistema registra evento e redireciona corretamente.

Fase 2 — Geração de QR Image

Objetivo:

Gerar PNG/SVG do QR.

Tarefas:

- instalar biblioteca de QR;
- criar serviço generateQrCode;
- salvar QR no Storage;
- exibir preview no painel.

Critério de aceite:

Usuário interno consegue gerar QR e testar com celular real.

Fase 3 — Print Asset Engine

Objetivo:

Aplicar QR real na arte final.

Tarefas:

- instalar sharp;
- criar templates de posição;
- criar serviço de composição;
- cobrir QR fictício, se existir;
- aplicar QR real;
- exportar PNG final;
- salvar no Storage;
- permitir download.

Critério de aceite:

Sistema gera arte final com QR real, legível e funcional.

Fase 4 — Dashboard Metrics

Objetivo:

Mostrar métricas para anunciante e operação.

Tarefas:

- criar views ou queries;
- criar cards de métricas;
- criar gráfico por dia;
- criar tabela de scans;
- adicionar filtros por período.

Critério de aceite:

Dashboard mostra scans reais do QR em tempo quase real.

Fase 5 — Placement-Level Tracking

Objetivo:

Rastrear por veículo/installação.

Tarefas:

- vincular qr_codes a placement/vehicle;
- gerar QR por veículo;
- gerar múltiplas artes finais;
- dashboard por veículo/motorista.

Critério de aceite:

Campanha consegue mostrar quais veículos geraram mais scans.

23. Checklist de QA

23.1. QR

- [] QR abre no celular.
- [] QR registra scan.

- [] QR redireciona corretamente.
- [] QR não expira.
- [] QR inativo não quebra.
- [] QR encerrado usa fallback.
- [] QR com WhatsApp abre mensagem correta.
- [] QR impresso em tamanho real é legível.

23.2. Tracking

- [] Registra campaign_id.
- [] Registra qr_code_id.
- [] Registra placement_id se houver.
- [] Registra user_agent.
- [] Registra device_type.
- [] Registra browser/os.
- [] Registra ip_hash sem salvar IP bruto.
- [] Marca bots básicos.
- [] Não quebra se user_agent vier vazio.

23.3. Arte

- [] Arte base permanece intacta.
- [] QR fictício é coberto corretamente.
- [] QR real fica na posição correta.
- [] QR tem fundo branco.
- [] QR tem margem de segurança.
- [] Arte final exporta em alta qualidade.
- [] Preview no painel corresponde ao arquivo final.
- [] Download funciona.

23.4. Dashboard

- [] Mostra total de scans.
- [] Mostra scans únicos estimados.
- [] Mostra gráfico por dia.
- [] Permite filtro por período.
- [] Não inclui bots por padrão.
- [] Métricas batem com eventos brutos.

24. Edge Cases

24.1. Campanha pausada

Comportamento:

Registrar scan e redirecionar para fallback ou página de campanha pausada.

24.2. Campanha encerrada

Comportamento:

Registrar scan e redirecionar para página segura.
Nunca retornar 404 seco.

24.3. Destino inválido

Comportamento:

Não permitir ativação do QR.
Mostrar erro no painel interno.

24.4. QR escaneado várias vezes pelo mesmo usuário

Comportamento:

Registrar eventos brutos.
Calcular unique scan por ip_hash + user_agent + janela de tempo.

24.5. Arte com tamanho diferente

Comportamento:

Template deve validar canvas.
Se tamanho não bater, exigir ajuste manual ou template compatível.

25. Nomes recomendados para módulos

Adaptar ao padrão do projeto.

```
/services/qr/qr-code.service.ts  
/services/qr/qr-tracking.service.ts  
/services/qr/qr-redirect.service.ts  
/services/assets/print-asset.service.ts  
/services/assets/layout-template.service.ts  
/services/metrics/qr-metrics.service.ts
```

Rotas possíveis:

```
/app/q/[slug]/route.ts
/app/api/qr-codes/route.ts
/app/api/campaign-assets/generate-final/route.ts
/app/api/campaigns/[id]/qr-metrics/route.ts
```

Se o projeto não for Next.js, adaptar ao framework real.

26. Variáveis de Ambiente

Adicionar somente se não existirem.

```
PUBLIC_APP_URL=https://driverads.com.br
QR_BASE_URL=https://driverads.com.br/q
IP_HASH_SALT=uma_string_secreta_forte
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
```

Atenção:

```
SUPABASE_SERVICE_ROLE_KEY nunca deve ir para o browser.
```

27. Padrão de Eventos

Se já existir tabela de eventos genérica, considerar registrar também nela.

Exemplo de evento:

```
{
  "event_type": "qr_scan",
  "campaign_id": "uuid",
  "qr_code_id": "uuid",
  "placement_id": "uuid",
  "source": "driverads_qr",
  "metadata": {
    "slug": "diniz-lentes-a7k2",
    "device_type": "mobile",
    "redirected_to": "https://wa.me/..."
  }
}
```

Se a plataforma já tem event tracking, talvez seja melhor reaproveitar a tabela existente e criar apenas `qr_codes`, evitando `qr_scan_events`.

Critério:

Se a tabela atual de eventos suporta volume alto, filtros por campanha e metadata, reaproveitar.
Se não suporta, criar `qr_scan_events` dedicada.

28. Importante: Não Criar Frankenstein

Antes de criar qualquer nova tabela, responder:

Essa entidade já existe?
Essa informação já é armazenada em outro lugar?
Existe relacionamento equivalente?
Existe padrão de eventos já usado?
Existe padrão de assets já usado?
Existe padrão de status já usado?
Existe dashboard de campanhas já pronto?

Princípio:

Adicionar a feature ao modelo atual.
Não criar um segundo sistema paralelo.

29. Entrega Esperada do Codex

Ao final, entregar:

1. Migrations SQL necessárias.
2. Serviços de QR.
3. Endpoint público de redirect.
4. Geração de QR.
5. Composição de arte final.
6. Upload/download no Storage.
7. Dashboard básico de métricas.
8. Testes manuais documentados.
9. Checklist de validação para impressão.

30. Critérios Finais de Aceite

A feature estará pronta quando:

- [] Um admin consegue criar QR para uma campanha.
- [] O QR aponta para `https://driverads.com.br/q/{slug}`.
- [] O QR não expira.
- [] O acesso ao QR registra scan no banco.
- [] O acesso redireciona para WhatsApp/site/landing configurado.
- [] O dashboard mostra scans reais.
- [] O sistema consegue aplicar QR real numa arte base.
- [] A arte final pode ser baixada.
- [] O QR da arte final é legível em celular real.
- [] Campanhas encerradas não quebram QR impresso.
- [] O banco não recebeu tabelas duplicadas desnecessárias.

31. Prioridade Estratégica

Esta feature não deve ser tratada como “gerador de QR Code”.

Ela deve ser tratada como:

Sistema proprietário de mídia física rastreável offline-to-online.

O QR Code é apenas a porta de entrada.

O ativo real da Driver ADS é:

- rastreamento;
- controle de campanha;
- métricas para o anunciante;
- prova de entrega;
- conexão entre mídia física e atendimento digital;
- inteligência por campanha, veículo e período.

Isso transforma a Driver ADS de uma mídia impressa em carros para uma plataforma mensurável de mídia local em movimento.